



**Universidad
Europea**

UNIVERSIDAD EUROPEA DE ANDALUCÍA

ESCUELA POLITÉCNICA

**MÁSTER UNIVERSITARIO EN ANÁLISIS DE GRANDES
VOLÚMENES DE DATOS (BIG DATA)**

TRABAJO FIN DE MÁSTER

**DESARROLLO DE UN SISTEMA
ESCALABLE DE MICROSERVICIOS PARA EL
ANÁLISIS DE DATOS IOT**

BORIS RENEE PATERNINA PEREZ

Dirigido por

VICTOR GOMEZ GUIRADO

CURSO 2025 - 2026

TÍTULO: DESARROLLO DE UN SISTEMA ESCALABLE DE MICROSERVICIOS PARA EL
ANÁLISIS DE DATOS IOT

AUTOR: BORIS RENEE PATERNINA PEREZ

TITULACIÓN: MÁSTER UNIVERSITARIO EN ANÁLISIS DE GRANDES VOLÚMENES DE
DATOS (BIG DATA)

DIRECTOR DEL PROYECTO: VICTOR GOMEZ GUIRADO

FECHA: Septiembre de 2026

RESUMEN

El resumen tiene entre 150-250 palabras. Resumir consiste en ofrecer información sobre cómo, dónde, cuándo y por qué se aplica el proyecto. Se realiza al finalizar el trabajo.

Debe incluir:

- Resumen del problema planteado y las aportaciones más importantes del proyecto al respecto
- Indica si procede si el proyecto se realizó en colaboración con una empresa, indicando nombre (siempre que haya autorización expresa por la empresa), y el sector industrial o empresarial
- Principales resultados del proyecto (diseño de un sistema para... , desarrollo e implementación de una solución para... , análisis de... , modelo de...)
- Resume las conclusiones más importantes del trabajo realizado
- El resumen NO
- Da una información genérica
- Se refiere a datos aportados en el texto del proyecto.

Palabras clave: hasta un máximo de 6 conceptos (un concepto puede conllevar una o más palabras). Incluye tecnologías que hayas utilizado, conceptos relevantes del ámbito científico-técnico, y conceptos de la industria (algunos ejemplos: machine learning, Big Data, Smart City, visión artificial, social media, Twitter, computación afectiva, transformación digital, GDPR, DevOps, EEG, ...)

ABSTRACT

Resumen en inglés.

Keywords: Palabras clave en inglés

Índice general

1. RESUMEN DEL PROYECTO	8
1.1. Contexto y justificación	8
1.2. Planteamiento del problema	8
1.3. Objetivos del proyecto	8
1.4. Resultados obtenidos	8
1.5. Estructura de la memoria	8
2. ANTECEDENTES / ESTADO DEL ARTE	9
2.1. Antecedentes	9
2.1.1. El crecimiento de la analítica de datos en el Internet de las Cosas	9
2.1.2. Categorización de la analítica de datos IoT	9
2.1.3. Arquitecturas de referencia: Lambda y Kappa	10
2.2. Estado del arte	12
2.2.1. Ecosistema de herramientas de código abierto en el ámbito de los datos IoT	12
2.2.2. Retos abiertos: seguridad, interoperabilidad y especialización técnica	14
2.2.3. Criterios de selección de herramientas de procesamiento streaming	14
2.3. Contexto y justificación	15
2.4. Planteamiento del problema	16
3. OBJETIVOS	18
3.1. Alcance del proyecto	18
3.2. Tareas a desarrollar	18
3.3. Objetivos	18
3.3.1. Objetivo 1: Garantizar la ingesta fiable de telemetría en tiempo real	19
3.3.2. Objetivo 2: Garantizar la gobernanza y evolución controlada del esquema de datos	19
3.3.3. Objetivo 3: Procesar y enriquecer los datos en streaming con baja latencia	19
3.3.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica	19
3.3.5. Objetivo 5: Validar la escalabilidad y resiliencia del sistema	20
3.4. Beneficios del proyecto	20
4. DESARROLLO DEL PROYECTO	21
4.1. Planificación del proyecto	21
4.2. Descripción de la solución, metodologías y herramientas empleadas	21
4.3. Recursos requeridos	21
4.4. Presupuesto	21
4.5. Viabilidad	22
4.6. Resultados del proyecto	23

5. DISCUSIÓN	24
6. CONCLUSIONES	25
6.1. Conclusiones del trabajo	25
6.2. Conclusiones personales	25
7. FUTURAS LÍNEAS DE TRABAJO	26
Bibliografía	27
8. ANEXOS	29

Índice de Figuras

2.1. Categorías, enfoques y componentes de la analítica de datos IoT	10
2.2. Arquitectura Lambda	11
2.3. Arquitectura Kappa	11
4.1. Esta es una imagen de ejemplo.	22

Índice de Tablas

2.1. Principales brokers MQTT con licencia aprobada por la Open Source Initiative .	12
2.2. Principales plataformas de mensajería persistente con licencia aprobada por la Open Source Initiative	13
4.1. Costes del proyecto	22

Capítulo 1. RESUMEN DEL PROYECTO

Este capítulo es un resumen, y no debe incluir el detalle. Existen capítulos específicos para añadir detalle de cada apartado. por tanto, únicamente se hace un resumen del trabajo completo, siguiendo la estructura marcada por las secciones de este capítulo.

Ocupa un máximo de 1 página.

1.1 Contexto y justificación

Resumen del por qué del proyecto y del contexto en el que se desarrolla. Está relacionado con el capítulo 2.

1.2 Planteamiento del problema

Resumen del planteamiento del problema que pretendemos solucionar con nuestro proyecto. Se busca una pregunta motriz a la que el trabajo pretende dar respuesta. Aquí se puede hablar del estado del arte o antecedentes de dicho problema.

Indica si el proyecto se ha desarrollado para resolver un problema específico empresarial, o para investigar en una cuestión específica del ámbito científico-técnico, y/o incluye de innovación.

1.3 Objetivos del proyecto

Incluye aquí solo un resumen de objetivos. En el capítulo correspondiente a objetivos se detallan tanto el general como los específicos.

1.4 Resultados obtenidos

Incluye el resumen de los resultados obtenidos al final de tu proyecto. En el capítulo correspondiente a resultados se detallan estos resultados.

1.5 Estructura de la memoria

Describe muy brevemente los capítulos y su contenido, con tus propias palabras.

Capítulo 2. ANTECEDENTES / ESTADO DEL ARTE

2.1 Antecedentes

2.1.1. El crecimiento de la analítica de datos en el Internet de las Cosas

El despliegue masivo de dispositivos conectados ha convertido a la analítica de datos en el elemento central para transformar la telemetría generada por sensores e infraestructuras inteligentes en información accionable. Como señala Y. Liu [1], el crecimiento exponencial del volumen, la velocidad y la variedad de los datos generados por miles de millones de dispositivos ha superado la capacidad de los enfoques tradicionales de procesamiento orientado a lotes, obligando a la creación y adopción de arquitecturas capaces de ingerir, procesar y almacenar flujos continuos de datos en tiempo real.

Junto a los factores tecnológicos de la transición impulsada por el crecimiento exponencial de los datos IoT, debe considerarse que el valor de dichos datos depende de la latencia con la que estos se procesan. Por ejemplo, los datos de temperatura de un motor industrial pierden gran parte de su utilidad si se analizan horas después de haber sido generados, cuando la ventana de acción preventiva ya se ha cerrado. Este factor tiempo motiva la centralidad que ha adquirido el procesamiento por flujos (*stream processing*) versus el procesamiento por lotes clásico para los pipelines IoT modernos.

El proceso de análisis de datos IoT abarca la recolección, el procesamiento, el almacenamiento y la extracción de conocimiento, y su relevancia se ha extendido a dominios tan diversos como la salud, las ciudades inteligentes, la automatización industrial, el monitoreo ambiental y el transporte inteligente (Y. Liu [1]).

2.1.2. Categorización de la analítica de datos IoT

Y. Liu [1] propone un marco de clasificación que resulta útil para situar cualquier pipeline dentro del panorama general de la analítica de datos IoT. El marco de clasificación distingue tres dimensiones globales, tal como se muestra en la Figura 2.1:

- **Categorías de análisis**, según el momento y el propósito del procesamiento: análisis descriptivo (qué ha ocurrido), análisis en tiempo real (qué está ocurriendo), análisis predictivo (qué es probable que ocurra) y análisis prescriptivo (qué se debería hacer al respecto).
- **Enfoques de despliegue**: procesamiento centralizado en la nube (*cloud-based*), procesamiento en el extremo de la red (*edge*) y procesamiento intermedio (*fog computing*), cada uno con distintos compromisos entre latencia, ancho de banda y capacidad de cómputo disponible.
- **Componentes funcionales**: ingesta y recolección de datos, almacenamiento y gestión, preprocesamiento y transformación, y finalmente analítica y aprendizaje automático.

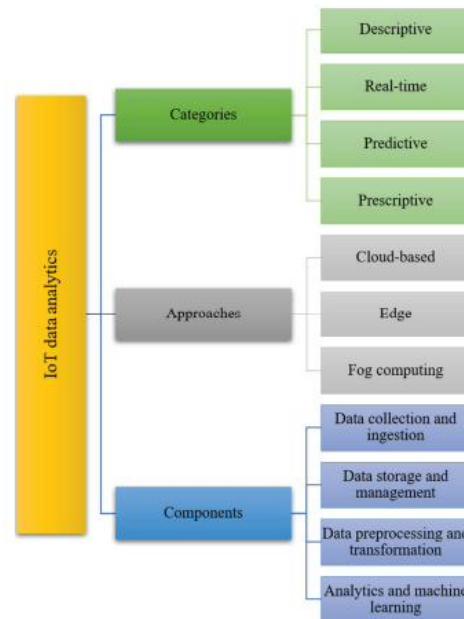


Figure 1 Overview of IoT data analytics categories, approaches, and components.

Figura 2.1. Categorías, enfoques y componentes de la analítica de datos IoT. Reproducida de Y. Liu [1], distribuida bajo licencia Creative Commons (CC BY).

2.1.3. Arquitecturas de referencia: Lambda y Kappa

En la literatura sobre procesamiento de grandes volúmenes de datos en streaming destacan dos patrones arquitectónicos ampliamente documentados. La arquitectura Lambda fue propuesta originalmente por N. Marz [2] en 2011, y posteriormente detallada junto con J. Warren [3]; parte de la premisa de que toda consulta puede expresarse como una función sobre la totalidad de los datos ($query = function(all\ data)$), pero calcular esa función en tiempo real sobre el dataset completo resulta prohibitivamente costoso. Para resolverlo, combina una capa de procesamiento por lotes (que recalcula vistas precisas sobre el histórico completo) con una capa de velocidad (que ofrece vistas aproximadas de baja latencia sobre los datos recientes), fusionadas después en una capa de servicio.

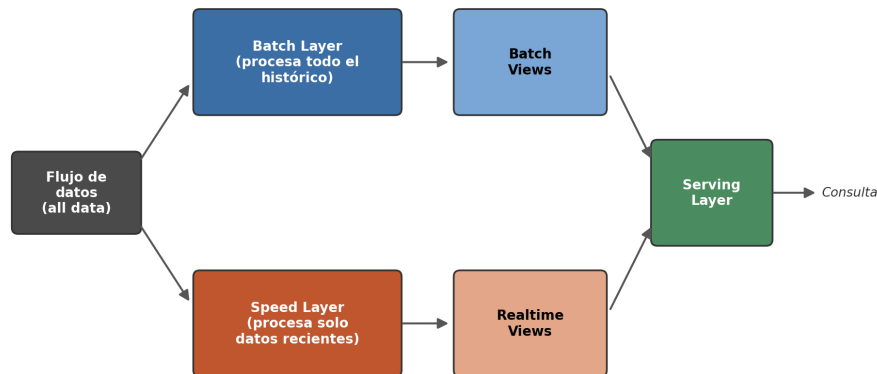


Figura 2.2. Arquitectura Lambda: las capas batch y speed se fusionan en la capa de servicio. Elaboración propia a partir de [2] y [3].

Aunque esta arquitectura fue ampliamente adoptada, J. Kreps [4] señala que esta exige implementar y mantener la misma lógica de transformación por duplicado: una en el sistema batch y otra en el de streaming; y sincronizar ambas transformaciones resulta muy costoso operativamente. H. Cao y M. Wachowicz [5] añaden que la dependencia de infraestructura de cómputo intensivo hace esta arquitectura poco adecuada para muchos escenarios IoT.

Como alternativa, J. Kreps [4] propuso en 2014 la arquitectura Kappa, que elimina la capa de lotes y utiliza un flujo reproducible (*replayable stream*) como fuente de verdad única. Cuando hace falta reprocesar el flujo (por un cambio de lógica o corrección de un bug), se lanza una segunda instancia del job de streaming (con una versión mejorada del código, corriendo sobre el mismo framework, con los mismos datos de entrada) que se ejecuta reproduciendo el flujo desde el principio del log retenido, escribiendo su resultado en una nueva tabla de salida y en paralelo a la que ya está en producción; una vez que el segundo job se sincroniza al presente, la aplicación se redirige a leer de la tabla nueva, y solo entonces se puede retirar la tabla y el job antiguos [4].

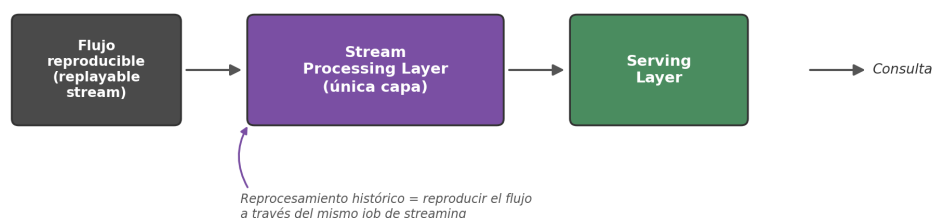


Figura 2.3. Arquitectura Kappa: una única capa de procesamiento de streaming, con reprocesamiento histórico mediante reproducción del flujo. Elaboración propia a partir de [4].

Ejemplos recientes de implementación práctica de Kappa en contextos IoT incluyen a S.

Shinde [6], quien construye un pipeline de monitorización de temperatura y detección de anomalías con Kafka, Spark Streaming y Streamlit, y a G. Zefkilis [7], que documenta una variante con Kafka, Spark Streaming, StarRocks y MinIO sobre Docker. Cabe señalar, no obstante, que ambos son tutoriales técnicos de un único autor.

2.2 Estado del arte

2.2.1. Ecosistema de herramientas de código abierto en el ámbito de los datos IoT

El ecosistema de herramientas de código abierto disponibles para cada etapa del pipeline ha madurado considerablemente, como ilustran las implementaciones de S. Shinde [6] y G. Zefkilis [7] comentadas en la sección anterior.

En la etapa de ingesta tenemos varios estándares abiertos disponibles para los protocolos de comunicación con los dispositivos IoT; entre ellos **MQTT** (ISO/IEC 20922) ofrece un modelo de publicación-suscripción por tópicos con broker central y niveles de calidad de servicio configurables, adecuado para dispositivos numerosos y de conectividad intermitente; **CoAP** sigue un modelo de petición-respuesta sobre UDP, pensado para dispositivos de recursos muy limitados, y solo incorpora publicación-suscripción mediante extensiones; **AMQP 1.0** (ISO/IEC 19464) está orientado a mensajería empresarial con mayores garantías y coste de implementación; y **DDS** adopta un modelo descentralizado sin broker, propio de sistemas de control en tiempo real.

Estableciendo MQTT como protocolo para este trabajo, los brokers disponibles para la recolección de telemetría se resumen en la Tabla 2.1.

Broker	Licencia	Lenguaje	Observaciones
Eclipse Mosquitto	EPL 2.0 / EDL 1.0	C	Implementación de referencia de la Eclipse Foundation; exclusivamente MQTT, de huella mínima
HiveMQ Community Ed.	Apache 2.0	Java	La extensión de integración con Kafka es producto comercial
VerneMQ	Apache 2.0	Erlang	Orientado a despliegues distribuidos, con clustering nativo
NanoMQ	MIT	C	Diseñado para el borde de la red
Apache ActiveMQ Artemis	Apache 2.0	Java	Multiprotocolo: AMQP 1.0, MQTT 3.1.1 y 5.0, y STOMP
RabbitMQ	MPL 2.0	Erlang	Broker de colas con soporte de MQTT 3.1, 3.1.1 y 5.0

Tabla 2.1. Principales brokers MQTT con licencia aprobada por la Open Source Initiative

De este inventario conviene excluir EMQX, habitualmente citado entre las opciones de licencia libre: desde su versión 5.9.0 el proyecto abandonó la licencia Apache 2.0 en favor de la Business Source License 1.1 [8], de carácter *source-available* y no aprobada por la Open Source Initiative.

Del lado del almacenamiento ordenado y reproducible del flujo en streaming que alimentará a la capa de procesamiento, la oferta de plataformas de mensajería persistente es igualmente amplia, como recoge la Tabla 2.2.

Plataforma	Licencia	Lenguaje	Observaciones
Apache Kafka	Apache 2.0	Java/Scala	Registro particionado con retención configurable y desplazamiento por consumidor
Apache Pulsar	Apache 2.0	Java	Separa cómputo y almacenamiento mediante BookKeeper; multi-tenencia y geo-replicación nativas
Apache RocketMQ	Apache 2.0	Java	Reproducción de mensajes por tiempo o por desplazamiento; ecosistema concentrado en Asia
NATS (JetStream)	Apache 2.0	Go	Binario único sin dependencia de JVM; ecosistema de conectores más reducido
RabbitMQ Streams	MPL 2.0	Erlang	Registro persistente añadido sobre un broker de colas

Tabla 2.2. Principales plataformas de mensajería persistente con licencia aprobada por la Open Source Initiative

Todas las plataformas de la Tabla 2.2 son técnicamente viables para una arquitectura Kappa, puesto que ofrecen registro persistente, reproducción por desplazamiento y consumidores duraderos; requisitos fundamentales que exige todo diseño.

Para el acoplamiento entre los brokers de recolección y las plataformas de mensajería persistente, no existe una solución de código abierto integrada en el propio broker: los que ofrecen un puente nativo lo hacen desde una edición comercial [9] o bajo licencias *source-available* que no cumplen la definición de la Open Source Initiative [8]. En consecuencia, toda arquitectura restringida a herramientas de código abierto debe incorporar un componente adicional de *bridging*. Este puede ser un conector genérico, como el conector MQTT de Stream Reactor para Kafka Connect [10], o un conector desarrollado a la medida si se quiere asignar funcionalidades adicionales a dicho componente.

En la etapa de procesamiento de flujos, Apache Flink se ha consolidado como la mejor opción para el cómputo en tiempo real usando la semántica de eventos, mientras que Kafka Streams ocupa un nicho más acotado como librería embebida para aplicaciones Java/Scala nativas de Apache Kafka que no requieren un clúster de procesamiento independiente. Apache Spark Structured Streaming, por su parte, se ha consolidado como el estándar de la industria para el procesamiento por micro-lotes, apoyado en la amplitud de su ecosistema y la unificación del procesamiento por lotes y en streaming bajo una misma API; un rasgo que comparte Apache Flink pero que en Spark se beneficia de una integración más amplia con el ecosistema de almacenamiento *lakehouse* y herramientas SQL (S. Lakshmipathy [11]).

Para el almacenamiento de series temporales de los datos IoT, sistemas especializados como InfluxDB [12], TimescaleDB [13] y QuestDB [14] compiten por ofrecer el mejor equilibrio entre velocidad de escritura, compresión de datos y capacidad de consulta analítica; conviene advertir, sin embargo, que buena parte de los benchmarks de rendimiento publicados provienen directamente de los creadores de cada producto como se aprecia en [12], [13] y [14],

por lo que dichos benchmarks deben interpretarse con cautela hasta que existan estudios independientes.

En la etapa de visualización, Grafana se ha convertido en el estándar para paneles operacionales sobre datos de series temporales, en muchos casos combinado con Prometheus para monitoreo de infraestructura.

T. Domínguez-Bolaño et al. [15] complementan la panorámica anterior con un análisis comparativo de plataformas IoT de código abierto de propósito más amplio (como ThingsBoard, openHAB, OpenRemote o SiteWhere) que integran varias de estas etapas en un único producto, frente al enfoque modular de ensamblar componentes especializados por capa que caracteriza a los pipelines contruidos a medida. Ambos enfoques (plataforma integrada y ensamblaje modular de componentes) representan compromisos distintos entre rapidez de despliegue y flexibilidad de adaptación ante los requisitos específicos de un proyecto.

En conjunto, la evidencia revisada sugiere que las herramientas de código abierto en el ámbito de los datos IoT son hoy suficientemente maduras para competir con las ofertas propietarias en la nube en la mayoría de los casos de uso, a costa de una mayor carga operativa que exige experiencia técnica para configurar, escalar y asegurar cada componente del pipeline.

2.2.2. Retos abiertos: seguridad, interoperabilidad y especialización técnica

Pese a la madurez alcanzada por las herramientas disponibles, Y. Liu [1] y T. Domínguez-Bolaño et al. [15] plantean, de forma independiente, tres retos que permanecen abiertos:

- En primer lugar, la seguridad: la superficie de ataque se amplía con cada dispositivo IoT conectado, muchos de los cuales carecen de mecanismos de seguridad inherentes debido a la naturaleza distribuida de las redes IoT y a la ausencia de protocolos de seguridad estandarizados entre fabricantes [1].
- En segundo lugar, la interoperabilidad: la coexistencia de múltiples protocolos, formatos de datos y estándares entre fabricantes sigue dificultando la integración de dispositivos heterogéneos dentro de una misma plataforma [15].
- En tercer lugar, la especialización técnica requerida: la complejidad y la escala de los datos generados por los dispositivos IoT hacen necesario el uso de herramientas o plataformas especializadas para procesarlos de forma efectiva [1]. El manejo de estas herramientas puede exigir un nivel de dominio técnico que no siempre está disponible en equipos de tamaño reducido.

2.2.3. Criterios de selección de herramientas de procesamiento streaming

La elección de un sistema concreto para la etapa de procesamiento de flujos dentro de una arquitectura Kappa no tiene una respuesta universal: distintas herramientas optimizan frente a distintos compromisos, y la decisión debe fundamentarse en los criterios aplicables al proyecto de procesamiento de streaming, no en percepciones públicas o popularidad de una herramienta.

A continuación se plantean algunos criterios considerados y su aplicación al dominio del procesamiento de streaming en general:

- **Requisitos de latencia vs. cadencia de generación de datos:** si la frecuencia de emisión de los dispositivos es del orden de segundos a decenas de segundos, un motor de procesamiento por micro-lotes no introduce una degradación perceptible de la información generada; ya que la ventaja de latencia sub-segundo con un motor de streaming nativo solo es relevante cuando el caso de uso exige control reactivo o detección de eventos críticos en tiempo real.
- **Madurez del ecosistema y de las APIs en el lenguaje de implementación:** la profundidad de soporte de un lenguaje concreto (Java, Scala, Python) varía entre motores; un ecosistema con API nativa más madura reduce fricción de desarrollo y mantenimiento.
- **Unificación del modelo de procesamiento batch/streaming:** algunos motores permiten expresar lógica de procesamiento por lotes y en streaming bajo una misma API, lo que simplifica la arquitectura cuando ambos modos de procesamiento coexisten en el sistema.
- **Disponibilidad de talento y curva de adopción:** la disponibilidad de profesionales con experiencia previa en una tecnología concreta condiciona la viabilidad de mantenimiento y escalado de un sistema en producción a largo plazo.

Específicamente, en el dominio del procesamiento de streaming de dispositivos IoT industriales, la cadencia habitual de generación de eventos (normalmente en la escala de segundos) sitúa las exigencias de latencia del sistema por debajo del umbral en el que las ventajas de un motor de streaming nativo resultan necesarias. Por tanto para los requisitos de latencia vs. cadencia de generación de datos, un motor de procesamiento por micro-lotes con ventanas de agregación temporal ofrece garantías de tolerancia a fallos y semántica *exactly-once* equivalentes a las de un motor de streaming nativo, sin incurrir en complejidades operativas adicionales. Este análisis no excluye la idoneidad de los motores de streaming nativos en escenarios de mantenimiento predictivo / sistemas de control reactivo / detección de eventos críticos en tiempo real, donde la latencia resulta un factor determinante del diseño.

2.3 Contexto y justificación

El estado del arte revisado en la sección anterior pone de manifiesto un patrón recurrente: la mayor parte de las referencias sobre arquitecturas Kappa para IoT disponibles hoy (tutoriales de ingeniería, publicaciones de blog técnico, documentación de proyectos open-source) tienen un carácter fundamentalmente demostrativo. Su objetivo es ilustrar un concepto (por ejemplo, monitoreo de temperatura con Kafka, Spark Streaming y Streamlit, o un pipeline equivalente con StarRocks y MinIO), más que ofrecer una implementación completa y reproducible. Esta brecha entre la abundancia de ejemplos ilustrativos y la escasez de implementaciones reproducibles constituye la primera motivación de este trabajo: construir, documentar y poner a disposición pública un pipeline IoT completo, containerizado y basado exclusivamente en herramientas de código abierto, que sirva tanto como trabajo académico evaluable, como referencia técnica reutilizable por terceros.

El proceso de revisión de la literatura evidenció vacíos de información que este proyecto puede contribuir a complementar, al menos parcialmente, desde la práctica: no se encontró un trabajo independiente que documente el comportamiento de un broker MQTT de perfil ligero (como Eclipse Mosquitto) acoplado en la capa de ingesta a un microservicio de *bridging* dedicado hacia Kafka; combinación de elementos que la bibliografía consultada menciona de forma tangencial y sin evaluación.

La gobernanza de esquemas mediante Avro y un registro como Apicurio sí está documentada como buena práctica en arquitecturas de streaming de propósito general [3] (especialmente en contextos de flotas de dispositivos que evolucionan con el tiempo), pero no aparece integrada como componente explícito en ninguna de las implementaciones Kappa-IoT de código abierto revisadas ([6]; [7]) ni en los trabajos académicos consultados sobre analítica y arquitectura IoT ([1]; [15]; [5]). Incorporar esta gobernanza aporta un elemento diferenciador respecto a las implementaciones revisadas, permitiendo la evolución controlada del esquema de telemetría a medida que cambian los tipos de sensor o los campos capturados, y acercando este pipeline a las prácticas ya consolidadas en arquitecturas de streaming de propósito general.

El diseño de doble sumidero (dual sink: métricas agregadas por ventana temporal hacia TimescaleDB para consumo en Grafana, y eventos individuales hacia PostgreSQL para consumo analítico) responde a una necesidad práctica de separar el consumo operacional en tiempo casi real (monitorización de planta, alertas) del consumo analítico orientado a negocio (reporting, análisis histórico, cruces con otras fuentes corporativas). Este patrón de separación si está documentado en la industria de servicios gestionados propietarios; por ejemplo, el caso de AWS con la flota de vehículos de SOCAR, documentado por DoYeun Kim et al. [16], pero no aparece implementado en ninguna de las arquitecturas Kappa open-source revisadas, las cuales suelen asumir un único sumidero de salida.

En conjunto, estos elementos (implementación completa y reproducible, gobernanza de esquemas mediante Apicurio/Avro, y separación de sumideros operacional/analítico) justifican la elección de la arquitectura desarrollada en este trabajo no como una simple aplicación de una solución ya estudiada, sino como una propuesta que extiende la arquitectura Kappa de referencia hacia una dirección que la literatura actual documenta de forma incompleta.

2.4 Planteamiento del problema

De la revisión anterior se desprende que, si bien existen múltiples implementaciones ilustrativas de arquitecturas Kappa aplicadas a IoT, y sistemas equivalentes de gobernanza de esquemas y doble sumidero documentados en contextos de streaming de propósito general, ninguna implementación Kappa-IoT open-source revisada combina simultáneamente (i) un broker MQTT ligero en el extremo acoplado a un microservicio de *bridging* dedicado hacia Kafka; (ii) un registro de esquemas Avro que valide el formato de cada evento transmitido y su admisibilidad entre la capa de ingesta y la capa de procesamiento; y (iii) una estrategia de doble sumidero que separe el consumo de datos operacionales en tiempo real del consumo analítico.

La mayoría de los recursos formativos y de referencia disponibles para diseñar este tipo de pipelines dependen, en algún punto de la cadena, de servicios gestionados o plataformas propietarias (colas de mensajería en la nube, motores de streaming como servicio, o plataformas de ingesta IoT gestionadas como AWS IoT Core o Azure IoT Hub), lo que dificulta la adopción en escenarios que requieren control total sobre el despliegue, ausencia de dependencia de un proveedor concreto, o la posibilidad de ejecutar el pipeline en un entorno local (o on-premise) mediante contenedores.

Considerando lo anterior, el problema planteado en este trabajo es *la falta de una implementación de referencia, open-source y containerizada, de una arquitectura Kappa para IoT que integre bridging MQTT-Kafka en el edge, gobernanza de esquemas mediante Avro/Avrocurio, y una estrategia de doble sumidero que separe el consumo operacional del analítico.*

En el desarrollo del trabajo se utilizará un conjunto de datos de telemetría de consumo energético como caso de estudio, evaluando la viabilidad técnica de la arquitectura y dejando como elementos reutilizables el código, la configuración de despliegue y la documentación asociada.

Capítulo 3. OBJETIVOS

3.1 Alcance del proyecto

Este trabajo abarca el diseño, desarrollo y validación de un sistema desacoplado basado en microservicios para la ingesta, procesamiento, análisis y visualización de datos de dispositivos IoT, utilizando como caso de uso el análisis de la telemetría de contadores de consumo energético en edificios con mediciones de electricidad, agua fría y agua caliente.

3.2 Tareas a desarrollar

Para cubrir el alcance descrito, se plantean las siguientes tareas:

- Analizar el estado del arte en las herramientas, estrategias y retos actuales en el procesamiento y análisis de datos en streaming.
- Analizar las tecnologías open-source disponibles para el procesamiento y análisis de datos de dispositivos IoT.
- Diseñar e implementar un simulador de comunicación IoT para la transmisión de datos de telemetría.
- Diseñar e implementar una arquitectura basada en contenedores para la orquestación y ejecución de los siguientes microservicios:
 - Servicio de alta disponibilidad para la ingesta de datos, mediante un gestor de mensajería que acepte protocolos de telemetría estándar.
 - Servicio de filtrado y transformación de datos que alimente dos sumideros: una base de datos optimizada para series temporales con métricas ya agregadas por ventana, y una base relacional para eventos individuales con tablas de referencia.
 - Servicio de análisis que calcule índices de eficiencia energética, efectúe análisis operativo y detección de anomalías en datos de consumo.
- Diseñar un servicio de visualización mediante dashboards, para la visualización interactiva de los datos procesados.
- Evaluar el rendimiento del sistema mediante pruebas de estrés y de carga para validar su capacidad de escalabilidad.

3.3 Objetivos

Los objetivos concretos de este trabajo se definen a continuación, acompañados cada uno de un indicador clave de rendimiento (KPI) que permite verificar su cumplimiento y de las acciones necesarias para alcanzarlo. Los valores numéricos de cada KPI se han fijado a partir de rangos razonables de la práctica de ingeniería y de la literatura revisada en el estado del arte; se validarán y, si procede, se recalibrarán a partir de las pruebas piloto realizadas sobre la implementación.

3.3.1. Objetivo 1: Garantizar la ingesta fiable de telemetría en tiempo real

KPI: latencia inferior a 2 segundos para la ingesta del 95 % de los mensajes MQTT publicados; calculada desde la publicación del mensaje hasta su persistencia final, con una tasa de pérdida inferior al 0.1 %.

Acciones: desplegar Mosquitto como broker MQTT y un microservicio de bridge MQTT-Kafka encargado de reenviar los eventos ingeridos al tópico de Kafka correspondiente; definir el nivel de calidad de servicio (QoS) y la topología de tópicos; instrumentar el simulador MQTT y el bridge para medir la latencia extremo a extremo; ejecutar pruebas de ingesta sostenida con el simulador para verificar el cumplimiento del KPI.

3.3.2. Objetivo 2: Garantizar la gobernanza y evolución controlada del esquema de datos

KPI: el 100 % de los eventos ingeridos se validan correctamente contra el esquema Avro registrado; 0 % de fallos de deserialización ante cambios de esquema compatibles hacia adelante o hacia atrás.

Acciones: definir el esquema Avro inicial de telemetría; configurar las reglas de compatibilidad de esquema en Apicurio; diseñar y ejecutar al menos una prueba de evolución de esquema verificando que no se interrumpe el flujo ni se pierden mensajes; documentar los límites de la protección que ofrece el registro y las reglas adicionales que deban implementarse fuera de él.

3.3.3. Objetivo 3: Procesar y enriquecer los datos en streaming con baja latencia

KPI: latencia de procesamiento por micro-lote en Spark Structured Streaming inferior a 3 segundos tras el cierre de cada ventana; throughput sostenido de al menos 50 eventos por segundo sin acumulación de retraso entre lotes sucesivos (este umbral procede de la literatura de referencia y no del caso de uso en este trabajo).

Acciones: implementar el job de Spark Structured Streaming (DataFrame API) con ventanas de agregación temporal y *watermarking* para el tratamiento de datos tardíos; instrumentar métricas de throughput y de duración de cada micro-lote (*batch duration*) del job; ejecutar pruebas de carga incremental hasta identificar el punto de saturación y verificar el cumplimiento del KPI dentro del rango definido.

3.3.4. Objetivo 4: Ofrecer visualización diferenciada operacional y analítica

KPI: los dashboards de Grafana muestran datos con una latencia de refresco inferior a 5 segundos; se dispone de al menos 3 reportes en Power BI que cubran, respectivamente, consumo energético, eficiencia operativa y detección de anomalías.

Acciones: diseñar los dashboards operacionales de Grafana sobre TimescaleDB; conectar Power BI a los eventos enriquecidos de PostgreSQL; definir y calcular los índices de eficiencia energética requeridos por las tareas de análisis; validar la latencia de refresco de los dashboards.

3.3.5. Objetivo 5: Validar la escalabilidad y resiliencia del sistema

KPI: el sistema soporta al menos 500 sensores concurrentes con un throughput no menor del 80 % calculado en base a 100 sensores concurrentes; ante el fallo de un servicio individual, el sistema se recupera en menos de 60 segundos sin pérdida de datos.

Acciones: diseñar un plan de pruebas con volumen creciente de sensores simulados (100, 250, 500 y +600 sensores concurrentes); medir el throughput y la latencia bajo cada nivel de carga; simular la caída de un servicio (por ejemplo, un executor de Spark) y medir el tiempo de recuperación y la integridad de los datos gracias a la retención en Kafka; documentar los resultados obtenidos.

3.4 Beneficios del proyecto

El desarrollo de este sistema aporta, por un lado, un trabajo académico evaluable que documenta con rigor una implementación completa de una arquitectura Kappa para IoT basada en herramientas de código abierto, cerrando parcialmente los vacíos identificados en el estado del arte respecto a la gobernanza de esquemas y la separación de sumideros operacional y analítico.

Por otro lado, el pipeline resultante queda disponible como referencia replicable mediante contenedores y sin dependencia de proveedores propietarios, lo que facilita su adopción como punto de partida en proyectos similares de monitorización energética.

Capítulo 4. DESARROLLO DEL PROYECTO

4.1 Planificación del proyecto

Incluye un cronograma y una breve descripción de todas las actividades reales y su esfuerzo, que se han realizado en el proyecto. Algunos ejemplos de actividades son: estudio del arte, análisis de alternativas actuales, estudio de informes previos, análisis técnico de interfaces con sistemas externos, diseño del sistema, diseño de la validación y/o de pruebas, desarrollo de Software, realización de entrevistas, visualización de videos relacionados con el tema, reuniones mantenidas con personas de empresa y otras entidades, etc. Puedes reutilizar el formato del plan indicado en tu anteproyecto.

4.2 Descripción de la solución, metodologías y herramientas empleadas

Para un proyecto cuyo foco sea el ámbito científico y de investigación, aquí se describen las metodologías utilizadas para la realización del proyecto, incluye herramientas tecnológicas, programas de cálculo, dispositivos utilizados, modelos aplicados para el desarrollo del proyecto, descripción del análisis de datos cuyos resultados se presentarán en la sección de resultados, etc. Para un proyecto cuyo foco sea el desarrollo de producto, aquí se describe la solución, modelos analizados, diseños realizados, algoritmos aplicados, desarrollos implementados, etc. incluye herramientas tecnológicas, programas de cálculo, dispositivos utilizados, modelos aplicados para el desarrollo del proyecto, descripción del análisis de datos cuyos resultados se presentarán en la sección de resultados, etc. Puedes añadir una estructura en subsecciones si facilita la organización de esta sección.

4.3 Recursos requeridos

En este apartado debes enumerar los recursos que has utilizado para la ejecución del proyecto (recursos técnicos, dispositivos, material de laboratorio, asistencia de expertos, etc.). Si bien, en la sección donde se describe Metodología y Herramientas empleadas, se describen en detalle las mismas, en esta sección, únicamente debes enumerarlas.

4.4 Presupuesto

El Presupuesto supone la evaluación económica total del proyecto. No olvides que tu tiempo también vale dinero, no sólo hay que incluir el coste de los materiales empleados. A continuación, se muestra un ejemplo de resumen de presupuesto. Puedes añadir entradas a esta

tabla si lo necesitas, conforme a la naturaleza de tu trabajo. Todos los recursos requeridos indicados en el apartado anterior deberían estar recogidos en la tabla de costes, aunque hayan tenido coste cero.

Tipo de coste	Valor	Comentarios
Horas de trabajo en el proyecto	[Número de horas]	Añade entradas para otras personas que hayan participado en tu proyecto para contabilizar sus horas aproximadas.
Equipo técnico utilizado	€	Si el equipo no lo has tenido que adquirir, indica su valor aproximado en el mercado si lo tuvieras que adquirir nuevo. Utiliza una entrada en la tabla por cada equipo (desktop, servidor, etc.)
Software utilizado	€	Si el Software no lo has tenido que adquirir, indica su valor aproximado en el mercado si lo tuvieras que adquirir. Indica coste 0€ si es Software que no requiere pagar precio por licencia. Ejemplos: IDE de desarrollo, herramienta de análisis de datos, programa de diseño gráfico, etc.
Estudios e informes	€	Si has tenido que adquirir algún informe de consultora o de revista de investigación, indica el coste.
Materiales empleados	€	P. ej.: material de laboratorio, sensores, etc.

Tabla 4.1. Costes del proyecto



Figura 4.1. Esta es una imagen de ejemplo.

4.5 Viabilidad

Este apartado es opcional, pero es interesante que incluyas un breve análisis de viabilidad económica del proyecto (relación coste / beneficio), y un análisis de sostenibilidad a futuro del resultado de tu proyecto.

4.6 Resultados del proyecto

Conforme a los objetivos específicos, describe los resultados finales obtenidos. Para proyectos con un enfoque de desarrollo de producto, debes incluir el resultado de tu plan de pruebas. Puedes añadir una descripción de cambios durante el proyecto respecto a los objetivos iniciales, y comentarios que quieras incluir de cómo has desarrollado las distintas actividades.

Capítulo 5. DISCUSIÓN

Esta sección es más habitual en trabajos de tipo científico.^o de "investigación", donde uno presenta brevemente los resultados principales y los discute, pero también puede utilizarse en otro tipo de trabajos.

Puedes incluir secciones específicas para discutir cuestiones como: limitaciones del estudio, limitaciones de la tecnología empleada, cambios respecto a objetivos planteados inicialmente.

Puedes incluir respuestas a preguntas del tipo: ¿la metodología inicialmente pensada ha sido útil?, ¿he tenido que adaptarme a cambios a lo largo del proyecto?, ¿qué cambios han sido y cómo he adaptado el proyecto para poder manejar esos cambios?, ¿qué impacto ha tenido el resultado de mi proyecto?

Capítulo 6. CONCLUSIONES

6.1 Conclusiones del trabajo

Breve descripción objetiva del resultado en relación al objetivo general de tu proyecto.

6.2 Conclusiones personales

Describe tus impresiones y experiencia personal durante el desarrollo del proyecto, o destacar la importancia que tiene el tema para ti, lo que has aprendido, o la trascendencia que ha tenido para ti o para otros este proyecto.

Capítulo 7. FUTURAS LÍNEAS DE TRABAJO

La implementación actual del pipeline se ha desplegado íntegramente mediante Docker Compose, lo cual resulta adecuado para el alcance de este TFM (un entorno de desarrollo y validación en una única máquina), pero presenta limitaciones evidentes de cara a un despliegue productivo real: no ofrece escalado horizontal automático de los componentes con mayor carga (particiones de Kafka, ejecutores de Spark Structured Streaming), ni mecanismos nativos de auto-recuperación ante fallos de nodo.

Una línea de evolución natural del sistema es la migración de la orquestación hacia una distribución ligera de Kubernetes, manteniendo el resto de la arquitectura (Mosquitto, Kafka en modo KRaft, Apicurio, Spark Structured Streaming y los sinks duales TimescaleDB/PostgreSQL) sin cambios sustanciales, ya que todos los componentes están containerizados y son, en principio, portables a manifiestos de Kubernetes o a Helm charts.

Entre las distribuciones evaluadas como candidatas destacan dos por su bajo consumo de recursos y su idoneidad para escenarios de IoT y edge computing:

- **K3s** (Rancher/SUSE): una distribución certificada por la CNCF (Cloud Native Computing Foundation), lo que garantiza compatibilidad completa con la API estándar de Kubernetes, que sustituye *etcd* por SQLite en despliegues de un solo nodo y reduce drásticamente el consumo de memoria frente a Kubernetes estándar, lo que la convierte en una de las opciones más utilizadas en estudios comparativos de orquestación de contenedores para entornos de recursos limitados.
- **MicroK8s** (Canonical): distribución empaquetada como snap, con fuerte integración en el ecosistema Ubuntu (el mismo utilizado en el entorno de desarrollo de este proyecto) y un sistema de complementos (*addons*) que simplifica la activación de componentes como almacenamiento persistente o ingress.

Como trabajo futuro, se propone:

1. Migrar el stack de Docker Compose a manifiestos de Kubernetes (o Helm charts), comenzando por un clúster de un solo nodo sobre K3s o MicroK8s en el mismo entorno WSL/Ubuntu ya utilizado para el desarrollo.
2. Evaluar el escalado horizontal de los ejecutores de Spark Structured Streaming y de las particiones de Kafka bajo carga variable, midiendo consumo de CPU, memoria y latencia, replicando la metodología de estudios recientes que han comparado el rendimiento y la eficiencia de recursos de varias distribuciones ligeras de Kubernetes en escenarios de edge computing bajo esas mismas métricas [17] y de trabajos que han evaluado herramientas de orquestación de contenedores específicamente en dichos entornos [18].
3. Explorar la incorporación de mecanismos de auto-recuperación y despliegue continuo (GitOps) sobre el clúster resultante, como extensión natural del repositorio actual del proyecto.

Bibliografía

- [1] Y. Liu, «Data Analytics in the Internet of Things Era: Tools, Approaches, Challenges, and Solutions,» *Journal of ICT Standardization*, vol. 13, n.º 4, págs. 427-448, 2025. DOI: 10.13052/jicts2245-800X.1344.
- [2] N. Marz, *How to Beat the CAP Theorem*, Thoughts from the Red Planet blog post, 2011. dirección: <https://nathanmarz.com/blog/how-to-beat-the-cap-theorem.html>.
- [3] N. Marz y J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. 20 Baldwin Road, PO Box 761, Shelter Island, NY 11964, USA: Manning Publications Co., 2015, pág. 328, ISBN: 9781617290343.
- [4] J. Kreps, *Questioning the Lambda Architecture*, O'Reilly Radar blog post, 2014. dirección: <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.
- [5] H. Cao y M. Wachowicz, «An Edge-Fog-Cloud Architecture of Streaming Analytics for Internet of Things Applications,» *Sensors*, vol. 19, n.º 16, pág. 3594, 2019. DOI: 10.3390/s19163594.
- [6] S. Shinde, *Kappa Architecture Unveiled: Real-Time IoT Innovation and Hands-On Implementation*, Level Up Coding blog post, mar. de 2025. dirección: <https://levelup.gitconnected.com/kappa-architecture-unveiled-real-time-iot-innovation-and-hands-on-implementation-d8ff57f37167>.
- [7] G. Zefkilis, *Kappa Architecture in Action: From Sensors to Dashboards with Kafka, Spark Streaming, StarRocks, MinIO, and Docker*, Data Engineer Things blog post, 2025. dirección: <https://blog.dataengineerthings.org/kappa-architecture-in-action-from-sensors-to-dashboards-with-kafka-spark-streaming-starrocks-80492a509e1a>.
- [8] EMQ Technologies, *Data Integration*, Documentación oficial de EMQX Enterprise. El puente de datos hacia Kafka no forma parte de la edición de código abierto. Consultado en agosto de 2026, 2026. dirección: <https://docs.emqx.com/en/emqx/latest/data-integration/data-bridges.html>.
- [9] HiveMQ, *HiveMQ Enterprise Extension for Kafka*, Documentación oficial de producto. La extensión requiere licencia comercial (HiveMQ Professional o Enterprise); la Community Edition, bajo Apache 2.0, no la incluye. Consultado en agosto de 2026, 2026. dirección: <https://www.hivemq.com/products/extensions/hivemq-kafka-extension/>.
- [10] Lenses.io, *Stream Reactor: Kafka Connect connectors*, Repositorio del proyecto, licencia Apache 2.0. Incluye el conector fuente MQTT para Kafka Connect. Consultado en agosto de 2026, 2026. dirección: <https://github.com/lensesio/stream-reactor>.

- [11] S. Lakshmipathy, *Apache Flink vs Apache Kafka Streams vs Apache Spark Structured Streaming: Comparing Stream Processing Engines*, Onehouse blog post, abr. de 2025. dirección: <https://www.onehouse.ai/blog/apache-spark-structured-streaming-vs-apache-flink-vs-apache-kafka-streams-comparing-stream-processing-engines>.
- [12] InfluxData, *InfluxDB 3.0 vs OSS Benchmarks*, Documento oficial de benchmarking, 2025. dirección: <https://get.influxdata.com/rs/972-GDU-533/images/InfluxDB-3-0-vs-OSS-Benchmarks.pdf>.
- [13] TigerData, *TimescaleDB vs. InfluxDB*, Blog oficial de TigerData, creador de TimescaleDB, 2026. dirección: <https://www.tigerdata.com/blog/timescaledb-vs-influxdb-for-time-series-data-timescale-influx-sql-nosql-36489299877>.
- [14] QuestDB, *InfluxDB 3 Core Benchmarks: QuestDB Comparison*, Blog oficial de QuestDB, 2025. dirección: <https://questdb.com/blog/influxdb3-core-benchmarks/>.
- [15] T. Domínguez-Bolaño, O. Campos, V. Barral, C. J. Escudero y J. A. García-Naya, «An overview of IoT architectures, technologies, and existing open-source projects,» *Internet of Things*, 2022. DOI: 10.1016/j.iot.2022.100626.
- [16] D. Kim, S. Park, Y. Park, V. Falconer e Y. Yun, *How SOCAR Built a Streaming Data Pipeline to Process IoT Data for Real-Time Analytics and Control*, AWS Big Data Blog, nov. de 2022. dirección: <https://aws.amazon.com/blogs/big-data/how-socar-built-a-streaming-data-pipeline-to-process-iot-data-for-real-time-analytics-and-control/>.
- [17] H. Koziolok y N. Eskandani, «Lightweight Kubernetes Distributions: A Performance Comparison of MicroK8s, K3s, K0s, and Microshift,» en *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE 2023)*, New York, NY, USA: ACM, 2023, págs. 17-29. DOI: 10.1145/3578244.3583737.
- [18] I. Čilić, P. Krivić, I. Podnar Žarko y M. Kušek, «Performance Evaluation of Container Orchestration Tools in Edge Computing Environments,» *Sensors*, vol. 23, n.º 8, pág. 4008, 2023. DOI: 10.3390/s23084008.

Capítulo 8. ANEXOS

Sirven para incluir documentación complementaria (planos, circuitos, código, ficheros de configuración, especificaciones técnicas y hojas de características, fichas explicativas, resultado de encuestas, reglamentación y normativas requeridas, etc.).